GENOMIC SEQUENCE MATCHING ENGINE

Under The Supervision of

Eng. Ahmed Husseiny

Eng. Mohammed Salah

TOP-LEVEL MODULE CODE

```
module genomic_matcher_top #( parameter SEQ_LEN = 4,
    parameter CHAR_WIDTH = 2,
    parameter SCORE_WIDTH = 8)
(
    input  wire clk,rst, start,
    input  wire [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_a_in,
    input  wire [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_b_in,
    output wire signed [SCORE_WIDTH-1:0] final_score,
    output wire done
);

    wire load, index_rst,counter_enable;
    wire score_clear,score_enable,last_index ;

    wire [$clog2(SEQ_LEN)-1:0] current_index;
    wire [CHAR_WIDTH-1:0] char_a;
    wire [CHAR_WIDTH-1:0] char_b;
```

```
    fsm_controlF inst_fsm (
        .clk(clk), .rst(rst), .start(start), .last_index(last_index),
        .load(load), .index_rst(index_rst), .counter_enable(counter_enable),
        .score_clear(score_clear), .score_enable(score_enable), .done(done)
    );


    seq_regs #( .SEQ_LEN(SEQ_LEN), .CHAR_WIDTH(CHAR_WIDTH)
    ) inst_regs (
        .clk(clk), .rst(rst), .load(load), .seq_a_in(seq_a_in), .seq_b_in(seq_b_in),
        .index(current_index), .seq_a_char(char_a), .seq_b_char(char_b)
    );

    counter_indexing #( .seq_len(SEQ_LEN)
    ) inst_counter (
        .clk(clk), .rst(rst), .enable(counter_enable),
        .index_rst(index_rst),.index(current_index),
        .last_index(last_index)
    );

    comparator_score #( .data_width(CHAR_WIDTH), .score_width(SCORE_WIDTH)
    ) inst_score (
        .clk(clk), .rst(rst), .enable(score_enable), .clear(score_clear),
        .seq_a(char_a), .seq_b(char_b), .score(final_score)
    );

endmodule
```

VERIFICATION RESULTS

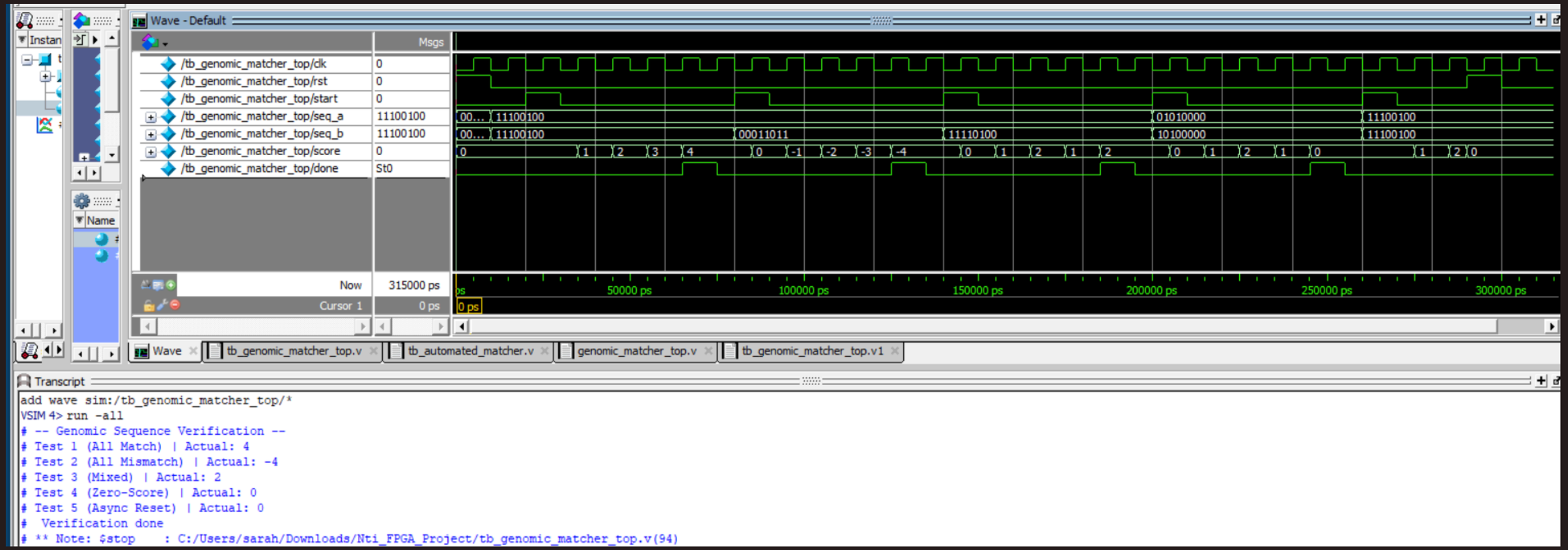
```
`timescale 1ns/1ps

module tb_genomic_matcher_top;
    parameter SEQ_LEN = 4;
    parameter CHAR_WIDTH = 2;
    parameter SCORE_WIDTH = 8;

    reg clk, rst, start;
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_a;
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_b;

    wire signed [SCORE_WIDTH-1:0] score;
    wire done;


    genomic_matcher_top #( .SEQ_LEN(SEQ_LEN), .CHAR_WIDTH(CHAR_WIDTH),
           .SCORE_WIDTH(SCORE_WIDTH)
     ) inst (
        .clk(clk), .rst(rst), .start(start),
        .seq_a_in(seq_a), .seq_b_in(seq_b),
        .final_score(score), .done(done)
    );
```

```
    initial begin

        clk = 0; rst = 1; start = 0;
        seq_a = 0; seq_b = 0;

        #10 rst = 0;
        $display("-- Genomic Sequence Verification --");

        // Test 1: All Match (ACGT vs ACGT),(Expected Score:+4)
        // A=00, C=01, G=10, T=11 (Binary packed: 11_10_01_00)
          seq_a = 8'b11_10_01_00;
          seq_b = 8'b11_10_01_00;
          #10 start = 1;
          #10 start = 0;
          wait(done == 1);
          $display("Test 1 (All Match) | Actual: %0d", score);
          #15;

        // Test 2: All Mismatch (ACGT vs TGCA),(Expected Score:-4)
        // seq_a: ACGT (11_10_01_00), seq_b: TGCA (00_01_10_11)
          seq_a = 8'b11_10_01_00;
          seq_b = 8'b00_01_10_11;
          start = 1;
          #10 start = 0;
          wait(done == 1);
          $display("Test 2 (All Mismatch) | Actual: %0d", score);
          #15;
```

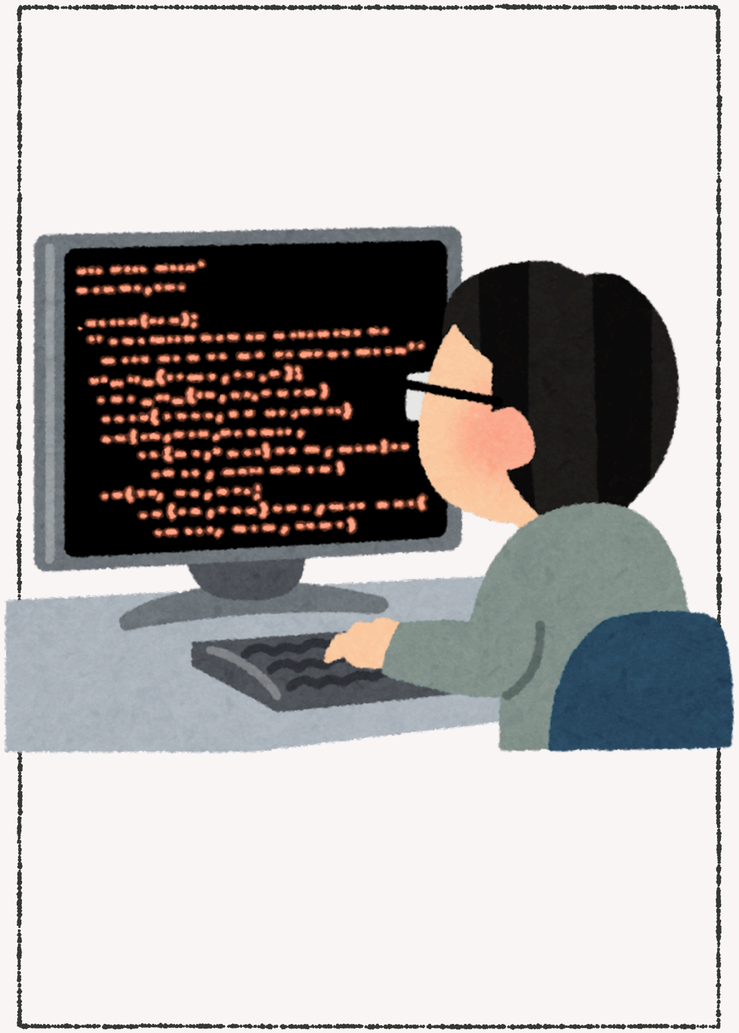
```
// Test 3: Mixed (ACGT vs ACTT), (Expected Score:+2)
// seq_a: ACGT (11_10_01_00), seq_b: ACTT (11_11_01_00)
  seq_a = 8'b11_10_01_00;
  seq_b = 8'b11_11_01_00;
  start = 1;
  #10 start = 0;
  wait(done == 1);
  $display("Test 3 (Mixed) | Actual: %0d", score);
  #15;

// Test 4: Zero-Score Scenario (2 Matches, 2 Mismatches)
// seq_a: CCAA (01_01_00_00), seq_b: GGAA (10_10_00_00)
  seq_a = 8'b01_01_00_00;
  seq_b = 8'b10_10_00_00;
  start = 1;
  #10 start = 0;
  wait(done == 1);
  $display("Test 4 (Zero-Score) | Actual: %0d", score);
  #15;
```

```
      // Test 5: Asynchronous Reset Mid-Transaction
      // Start a match, then hit reset before it finishes
        seq_a = 8'b11_10_01_00;
        seq_b = 8'b11_10_01_00;
        start = 1;
        #10 start = 0;

        // Wait 2 clock cycles to ensure FSM enter COMPARE state
        #20;

        rst = 1;
        #10;
        rst = 0;

        // Check system safely cleared the score (Expected: 0)
        $display("Test 5 (Async Reset) | Actual: %0d", score);
        #15;

        $display(" Verification done ");
        $stop;
    end
endmodule
```

WAVEFORM & TERMINAL



MATLAB IMPLEMENTION



CONFIGURATION PARAMETERS

```
num_tests = 50;
seq_len = 4;
score_width = 8;
```

SEQUENCE GENERATION & FILE HANDLERS

```
% Generate random sequences (0=A, 1=C, 2=G, 3=T)
seq_A = randi([0, 3], num_tests, seq_len);
seq_B = randi([0, 3], num_tests, seq_len);

% Open files for writing test vectors
fid_A = fopen('seq_a.mem', 'w');
fid_B = fopen('seq_b.mem', 'w');
fid_score = fopen('expected_scores.mem', 'w');
```

SCORING LOGIC & BINARY PACKING

```
for i = 1:num_tests
    score = 0;
    bin_str_A = '';
    bin_str_B = '';

    % Calculate score and build binary strings sequence by sequence
    for j = 1:seq_len
        % Build the packed 2-bit binary representation string
        bin_str_A = [bin_str_A, dec2bin(seq_A(i, j), 2)];
        bin_str_B = [bin_str_B, dec2bin(seq_B(i, j), 2)];

        % +1 for match, -1 for mismatch
        if seq_A(i, j) == seq_B(i, j)
            score = score + 1;
        else
            score = score - 1;
        end
    end
```

TWO'S COMPLEMENT & FILE EXPORT

```
    % Handle 2's complement for negative scores
    if score < 0
        twos_comp_score = (2^score_width) + score;
    else
        twos_comp_score = score;
    end

    % Write the binary strings to the .mem files
    fprintf(fid_A, '%s\n', bin_str_A);
    fprintf(fid_B, '%s\n', bin_str_B);
    fprintf(fid_score, '%s\n', dec2bin(twos_comp_score, score_width));
end

fclose(fid_A);
fclose(fid_B);
fclose(fid_score);

disp('Generated 50 test vectors successfully! Check for the .mem files.');
```

AUTOMATED MATCHER

```
`timescale 1ns/1ps

module tb_automated_matcher;
    parameter SEQ_LEN = 4;
    parameter CHAR_WIDTH = 2;
    parameter SCORE_WIDTH = 8;
    parameter NUM_TESTS = 50;

    reg clk, rst, start;
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_a;
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] seq_b;

    wire signed [SCORE_WIDTH-1:0] score;
    wire done;

    // Arrays to hold data read from MATLAB's files
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] mem_seq_a [0:NUM_TESTS-1];
    reg [(SEQ_LEN*CHAR_WIDTH)-1:0] mem_seq_b [0:NUM_TESTS-1];
    reg [SCORE_WIDTH-1:0] mem_expected_score [0:NUM_TESTS-1];

    genomic_matcher_top #( .SEQ_LEN(SEQ_LEN), .CHAR_WIDTH(CHAR_WIDTH),
        .SCORE_WIDTH(SCORE_WIDTH) ) inst (
        .clk(clk), .rst(rst), .start(start),
        .seq_a_in(seq_a), .seq_b_in(seq_b),
        .final_score(score), .done(done)
    );
```

```
    always #5 clk = ~clk;

    integer i;
    integer errors;
    reg signed [SCORE_WIDTH-1:0] expected;

    initial begin
        //Load
        $readmemb("seq_a.mem", mem_seq_a);
        $readmemb("seq_b.mem", mem_seq_b);
        $readmemb("expected_scores.mem", mem_expected_score);

        clk = 0; rst = 1; start = 0;
        errors = 0;

        #10 rst = 0;
        $display("-- Starting Automated Golden Model Verification --");

        for (i=0; i<NUM_TESTS; i=i+1) begin
            wait(done == 0);
            @(negedge clk);

            // Feed  randomized data into the hardware
            seq_a = mem_seq_a[i];
            seq_b = mem_seq_b[i];
            expected = mem_expected_score[i];

            start = 1;
            #10 start = 0;

            wait(done == 1);
```

```
    always #5 clk = ~clk;

    integer i;
    integer errors;
    reg signed [SCORE_WIDTH-1:0] expected;

    initial begin
        //Load
        $readmemb("seq_a.mem", mem_seq_a);
        $readmemb("seq_b.mem", mem_seq_b);
        $readmemb("expected_scores.mem", mem_expected_score);

        clk = 0; rst = 1; start = 0;
        errors = 0;

        #10 rst = 0;
        $display("-- Starting Automate

        for (i=0; i<NUM_TESTS; i=i+1) 
            wait(done == 0);
            @(negedge clk);

            // Feed  randomized data i
            seq_a = mem_seq_a[i];
            seq_b = mem_seq_b[i];
            expected = mem_expected_sc

            start = 1;
            #10 start = 0;

            wait(done == 1);
```

```
            //Compare Hardware output vs MATLAB expected output
            if (score !== expected) begin
                $display
                ("ERROR at test %0d: seq_a=%b, seq_b=%b | Expected: %0d, Actual: %0d",
                          i+1, seq_a, seq_b, expected, score);
                errors = errors + 1;
            end
        end

        //Print final results
        if (errors == 0)
            $display
         ("SUCCESS: All %0d randomized tests passed and matched MATLAB perfectly!", NUM_TESTS);
        else
            $display("FAILED: %0d errors found.", errors);

        $display("Verification Complete");
        $stop;
    end
endmodule
```

References

- Tomohiro, H. et al. (2014). "FPGA-Accelerator for DNA Sequence Alignment Based on an Efficient Data-Dependent Memory Access Scheme." Proceedings of the International Workshop on Highly Efficient Accelerators and Reconfigurable Technologies (HEART).

- Ali, K. et al. (2020). "Implementation of DNA sequence alignment algorithms using FPGA." International Journal of Advanced Computer Science and Applications.

- Needleman, S. B., & Wunsch, C. D. (1970). "A general method applicable to the search for similarities in the amino acid sequence of a protein." Journal of Molecular Biology, 48(3), 443–453.

THANKS!

ANY QUESTIONS?

Team Members

Alaa Maher Eldeeb - 213584

Nada Haron - 219628

Sarah Samir Ali - 232384

Rofida Mahmoud - 301873

Bassma Mohamed - 345332